

Projeto e Análise de Algoritmos - Lista 1

FGV - EMAP

Setembro de 2025

1. Encontre $h(n)$ de forma que $T(n) = O(h(n))$ para cada função a seguir, provando seu resultado:

(a) $T(n) = 3 \log n + n \cdot \sqrt{n} + 2025$

(b) $T(n) = 2^{n+2} + n + 33$

(c) $T(n) = 2^{2 \cdot n} + n^2$

(d) $T(n) = a_0 + a_1 \cdot n + a_2 \cdot n^2 + \dots + a_k \cdot n^k$ onde $a_k > 0$

2. Prove que $T(n) = n \cdot \log n$ é $O(n^{1+\delta})$ para todo $\delta > 0$.

3. Encontre e justifique a complexidade assintótica do seguintes trecho de código.

(a)

```
1 int find_something(int n, int v[]){
2     int l=0, r=n-1;
3
4     while(l<=r){
5         if(v[l]*v[r]<v[l]+v[r]+n){
6             l++;
7         }
8         else{
9             r--;
10        }
11    }
12
13    return min(l, r);
14 }
```

(b)

```
1 int something(int n) {
2     if (n == 0) return 0;
3
4     int val = 0;
5     for (int i = 0; i < n; i++){
6         val += ((n%i)==0);
7     }
8
9     for (int i = 0; i < 2; i++){
10        val += something(n/2);
11    }
12
13    return val;
14 }
```

4. Para cada item a seguir, resolva a recorrência para encontrar a função $f(n)$ tal que $T(n) = O(f(n))$. Em uma das questões, resolva a recorrência usando os quatro métodos vistos em aula.

$$(a) \quad T(n) = \begin{cases} 7 \cdot T(n/4) + 3n \log n + \sqrt{n} & \text{se } n > 1 \\ \Theta(1) & \text{se } n = 1 \end{cases}$$

$$(b) \quad T(n) = \begin{cases} 2 \cdot T(n/3) + 4n^3 & \text{se } n > 1 \\ \Theta(1) & \text{se } n = 1 \end{cases}$$

5. Mostre que:

$$(a) \quad T(n) = T(n-1) + T(n-2) + \Theta(1), \text{ com } T(1) = \Theta(1) \text{ é } O(2^n) \text{ e } \Omega(2^{n/2}).$$

Para os próximos exercícios você deve alterar o arquivo `lista-1-paa.py` adicionando a solução no corpo da função correspondente.

Atenção:

- Não altere o nome do arquivo Python.
- Para cada questão abaixo existe uma função definida no arquivo Python.
- Não altere a assinatura das funções. Você deve receber e retornar os tipos definidos.
- Caso queira definir uma classe ou função auxiliar, faça isso dentro da função do problema que estiver resolvendo.

6. Dada uma árvore binária de busca (BST) T com n nós contendo inteiros, projete um algoritmo que encontre a menor diferença absoluta entre dois nós diferentes da árvore. O algoritmo deve ter complexidade $O(n)$ no pior caso.

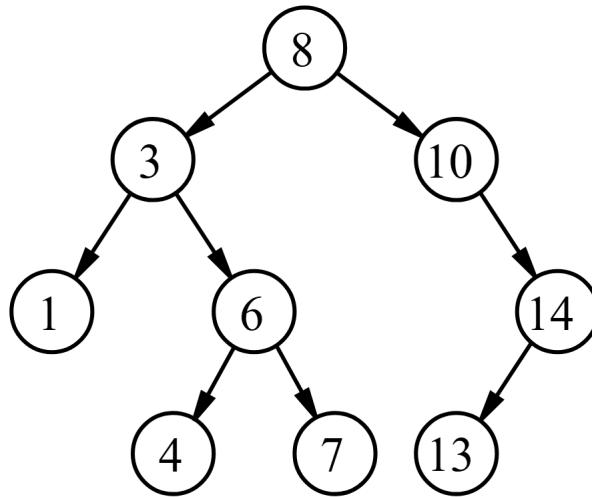


Figura 1: Exemplo de BST com menor diferença absoluta igual a 1 (vértices 6 e 7).

7. Desenvolva um algoritmo $O(n)$ que particiona um array A em números pares e ímpares. O algoritmo deve terminar com A contendo todos os seus elementos pares precedendo todos os seus elementos ímpares. A solução deve ser um algoritmo in-place, o que significa que ele pode usar apenas um espaço de memória constante além do próprio A . Na prática, isso significa que você não pode usar outro array auxiliar.

Exemplos:

Entrada	Saída
$A = [1, 2, 3, 4]$	$A = [2, 4, 3, 1]$
$A = [7, 17, 74, 21, 7, 9, 26, 10]$	$A = [74, 10, 26, 17, 7, 21, 9, 7]$

8. Dado um inteiro k e uma lista A que contém n inteiros, projete um algoritmo que retorne a quantidade de pares de inteiros em A cuja soma seja k . Mais especificamente, retorne a quantidade de pares (i, j) com $i < j$, tal que $A_i + A_j = k$.

Restrições:

- $A_i \leq 10^9$ para todo $1 \leq i \leq n$
- $k \leq 2 \cdot 10^9$
- $A_i \leq A_j \forall 1 \leq i < j \leq n$ nos itens (b) e (c)

Exemplos:

Entrada	Saída
$A = [3, 2, 6]$ e $k = 7$	0
$A = [1, 4, 5, 1, 3, 2, 6]$ e $k = 7$	4
$A = [1, 4, 5, 1, 3, 2, 6]$ e $k = 5$	3

- (a) Projete o algoritmo com complexidade de execução $O(n)$.
- (b) Agora assuma que A esteja ordenado ($A_i \leq A_j \ \forall \ i < j$), projete o algoritmo com complexidade de execução $O(n \log n)$ e complexidade de espaço $O(1)$.
- (c) Novamente assumindo que A está ordenado, projete o algoritmo com complexidade de execução $O(n)$ e complexidade de espaço $O(1)$.